Trust No Input: AI Assistance & Code-Comprehension Prompt Log

—

BreaQ 2026 — Challenge 1 Documentation of Conceptual & Repository Code Queries

Team: $\mu - Qbit$
Participants: Richard Watry, Dylane Mahazoasy
Date: October 25, 2026

October 25, 2026

Abstract

This document records the prompts used during Challenge 1 (*Trust No Input*) to clarify specific lines of Python and Qiskit code in the provided repository (`breaq_ch1/service.py` and `01_trust_no_input.ipynb`) as well as the underlying quantum-computing and mathematical concepts. All experimental design, vulnerability analysis, exploit construction, and mitigation implementation were performed independently by the team.

1 Scope and Methodology of AI Usage

During our exploration of the `breaq_ch1` codebase, we used an LLM strictly as an interactive documentation and code-comprehension reference when reading unfamiliar Qiskit idioms or Python constructs in `breaq_ch1/service.py`. Specifically, our prompts focused on:

1. Understanding how `build_bv_probe` constructs the Bernstein–Vazirani circuit and why the ancilla qubit is initialized to $|-\rangle$.
2. Understanding Qiskit's classical register bit-ordering convention and why `canonical_bits_from_counts` reverses the string with `[::-1]`.
3. Understanding the Python syntax in `_validate_map` and `compatibility_decode`.
4. Understanding the role of `transpile(..., optimization_level=0)` and `BasicSimulator` in `ChallengeService.execute`.
5. Reviewing background group-theoretic terminology for coordinate permutations acting on fixed-length bitstrings.

2 Repository Code Comprehension Prompts

2.1 Prompt 1 — Understanding `build_bv_probe` in `service.py` (Lines 27–49)

Context in Repository: When first reading `breaq_ch1/service.py`, we wanted to make sure we understood every gate in `build_bv_probe(fingerprint: str)` and why there are $n+1$ qubits but only n classical bits.

Listing 1: Lines 27–49 of `breaq_ch1/service.py`.

```
def build_bv_probe(fingerprint: str) -> QuantumCircuit:
    _validate_bits(fingerprint)
    n = len(fingerprint)
```

```
    qc = QuantumCircuit(n + 1, n, name="policy_probe")
    ancilla = n
    qc.x(ancilla)
    qc.h(ancilla)
    for qubit in range(n):
        qc.h(qubit)
    qc.barrier()
    for qubit, bit in enumerate(fingerprint):
        if bit == "1":
            qc.cx(qubit, ancilla)
    qc.barrier()
    for qubit in range(n):
        qc.h(qubit)
    qc.measure(range(n), range(n))
    return qc
```

Prompt Used:

> *"In Qiskit, I am reading this function* `build_bv_probe` *from the repo: it creates* `QuantumCircuit(n + 1, n)`*, applies* `qc.x(ancilla)` *then* `qc.h(ancilla)` *on* `ancilla = n`*, and inside the loop applies* `qc.cx(qubit, ancilla)` *when* `bit == "1"`*. Why isn't the ancilla qubit measured at the end, and how does* `cx(qubit, ancilla)` *transfer the bit* `"1"` *onto the input qubits?"*

Takeaway Used by the Team: This clarified the textbook Bernstein–Vazirani phase-kickback mechanism: applying X then H prepares the ancilla in $|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$, which is an eigenstate of X with eigenvalue -1. Consequently, each $\mathrm{CX}(q_i, \text{ancilla})$ kicks back a $(-1)^{s_i x_i}$ phase onto the control wire q_i without entangling the ancilla, and the final layer of Hadamards maps that phase directly to the computational basis state $|s_i\rangle$ upon measurement of qubits $0 \ldots n-1$.

2.2 Prompt 2 — Qiskit Endianness and `[::-1]` in `service.py` (Lines 52–59)

Context in Repository: In `breaq_ch1/service.py`, we noticed that `canonical_bits_from_counts` reverses the string returned by `dominant_key(counts)` using the slice `[::-1]`.

Listing 2: Lines 52–59 of `breaq_ch1/service.py`.

```
def dominant_key(counts: dict[str, int]) -> str:
    return max(counts, key=counts.get).replace(" ", "")


def canonical_bits_from_counts(counts: dict[str, int]) -> str:
    """Trusted q0..q(n-1) interpretation of a Qiskit count key."""
    return dominant_key(counts)[::-1]
```

Prompt Used:

> *"Can you explain lines 52–58 in* `service.py`*? First, what does* `max(counts, key=counts.get).replace(" ", "")` *do on a Qiskit counts dictionary? Second, why does* `canonical_bits_from_counts` *reverse the string with* `[::-1]` *when the docstring says 'Trusted q0..q(n-1) interpretation of a Qiskit count key'?"*

Takeaway Used by the Team: We learned that Qiskit formats classical register keys in little-endian order with the highest-indexed classical bit on the left ($c_{n-1} \ldots c_1 c_0$), and separates multiple classical registers with spaces. Passing `key=counts.get` selects the bitstring key with the largest shot count, and `[::-1]` reverses $c_3 c_2 c_1 c_0$ into index order $c_0 c_1 c_2 c_3$ so that string index `i` corresponds directly to qubit q_i.

2.3 Prompt 3 — Tuple Validation and Comprehension in `service.py` (Lines 20–24 & 61–65)

Context in Repository: We wanted to be certain of how `_validate_map` and `compatibility_decode` process the `compatibility_map` array from `team_instance.json`.

Listing 3: Lines 20–24 and 61–65 of `breaq_ch1/service.py`.

```
def _validate_map(order: Iterable[int], n: int = 4) -> tuple[int, ...]:
    order = tuple(int(i) for i in order)
    if sorted(order) != list(range(n)):
        raise ValueError("compatibility_map must be a permutation")
    return order

def compatibility_decode(counts: dict[str, int], compatibility_map: Iterable[int]) -> str:
    """Legacy reporting adapter. Intentionally unsafe for security decisions."""
    canonical = canonical_bits_from_counts(counts)
    order = _validate_map(compatibility_map, len(canonical))
    return "".join(canonical[index] for index in order)
```

Prompt Used:

> *"In `_validate_map` and `compatibility_decode`, how does `if sorted(order) != list(range(n))` guarantee that `order` is a valid permutation, and in what direction does `"".join(canonical[index] for index in order)` permute the characters of `canonical`?"*

Takeaway Used by the Team: Sorting any sequence of integers yields `[0, 1, ..., n-1]` if and only if the sequence has length n and contains each integer in $\{0, \ldots, n-1\}$ exactly once (i.e., a bijection on $\{0, \ldots, n-1\}$). Furthermore, `"".join(canonical[index] for index in order)` places the character `canonical[order[j]]` at output position `j` ($r_j = c_{\pi(j)}$), which helped us trace the exact index mapping by hand.

2.4 Prompt 4 — `transpile` and `BasicSimulator` in `service.py` (Lines 103–116)

Context in Repository: In `ChallengeService.execute`, the circuit is passed through `transpile(circuit, self._simulator, optimization_level=0)` before calling `self._simulator.run(...)`.

Listing 4: Lines 103–110 of `breaq_ch1/service.py`.

```
    def execute(self, case_id: str, shots: int = 512, seed: int = 2026) -> dict:
        case = self._cases[case_id]
        circuit = self.get_circuit(case_id)
        compiled = transpile(circuit, self._simulator, optimization_level=0)
        result = self._simulator.run(compiled, shots=shots, seed_simulator=seed).result()
        counts = dict(result.get_counts())
```

Prompt Used:

> *"In `ChallengeService.execute`, why does the code call `transpile(circuit, self._simulator, optimization_level=0)` before running on `BasicSimulator()`, and does `BasicSimulator` introduce any gate noise or readout error when `seed_simulator=2026` is set?"*

Takeaway Used by the Team: This confirmed that `BasicSimulator` is a noiseless ideal statevector/sampling simulator included in standard Qiskit, and `optimization_level=0` simply lowers high-level instructions if needed without rearranging or optimizing gates or permuting logical qubits. Therefore, for a deterministic Bernstein–Vazirani circuit, all 512 shots deterministically collapse to a single bitstring.

3 Conceptual & Mathematical Curiosity Prompts

3.1 Prompt 5 — Permutations of Bitstrings and Hamming Weight Preservation

Prompt Used:

> *"From a combinatorics and group-theory perspective, if a permutation $\pi \in S_n$ reorders the coordinates of an n-bit binary string $x \in \{0,1\}^n$, what properties of x are always invariant under π, and when can two different permutations π_1, π_2 produce the same output string on x?"*

Takeaway Used by the Team: Reordering the coordinates of a binary string preserves the number of ones (its Hamming weight $w(x)$) and number of zeros ($n - w(x)$). Moreover, if a bitstring has repeated bits at certain positions, swapping two positions that hold identical bit values leaves the bitstring unchanged (forming the stabilizer subgroup of x in S_n). This deepened our theoretical appreciation of how coordinate permutations act on $\{0,1\}^4$.